

Course-Style Project Specification

AI Document Assistant using Retrieval-Augmented Generation (RAG)

Project Overview

This project guides you through building a real-world AI system that combines large language models (LLMs) with information retrieval techniques. The system will allow users to upload documents (such as PDFs or text files) and ask questions about their contents. Instead of relying only on the language model's internal knowledge, the system retrieves relevant information from the uploaded documents and provides context-aware answers.

Learning Objectives

- Understand the architecture of Retrieval-Augmented Generation (RAG) systems.
- Implement document ingestion, chunking, and embedding pipelines.
- Build a semantic search system using a vector database (e.g., FAISS).
- Integrate a Large Language Model (LLM) for question answering.
- Deploy the system as an API using FastAPI.
- Document and present a working AI system similar to real-world AI products.

System Architecture

The system should follow the standard Retrieval-Augmented Generation pipeline: 1. User uploads documents. 2. Documents are parsed and split into smaller chunks. 3. Each chunk is converted into vector embeddings. 4. Embeddings are stored in a vector database. 5. When the user asks a question, the system retrieves the most relevant chunks. 6. Retrieved context is sent to the LLM. 7. The LLM generates a grounded answer using the retrieved context.

Recommended Tech Stack

- Python
- LangChain or LlamaIndex
- OpenAI API or open-source LLM
- FAISS (vector database)
- FastAPI for deployment
- Optional: Streamlit for a simple UI

Required Deliverables

- Working RAG system capable of answering questions from uploaded documents.
- FastAPI backend exposing at least one question-answer endpoint.

- Vector database containing embedded document chunks.
- GitHub repository with clean structure and documented code.
- README explaining system architecture and how to run the project.

Suggested Milestones

- Milestone 1: Document ingestion and chunking pipeline.
- Milestone 2: Embedding generation and FAISS vector store.
- Milestone 3: Retrieval pipeline using semantic similarity.
- Milestone 4: LLM integration for answer generation.
- Milestone 5: API deployment using FastAPI.
- Milestone 6: Final testing and documentation.

Evaluation Criteria (Self-Assessment)

- Correct implementation of the RAG pipeline.
- Quality of system architecture and modular design.
- Clarity and documentation of the GitHub repository.
- Quality of answers generated by the system.
- Successful deployment and demonstration.

Submission Checklist

- GitHub repository link
- Short demo video (optional but recommended)
- Project README with architecture diagram
- Instructions to run the system locally